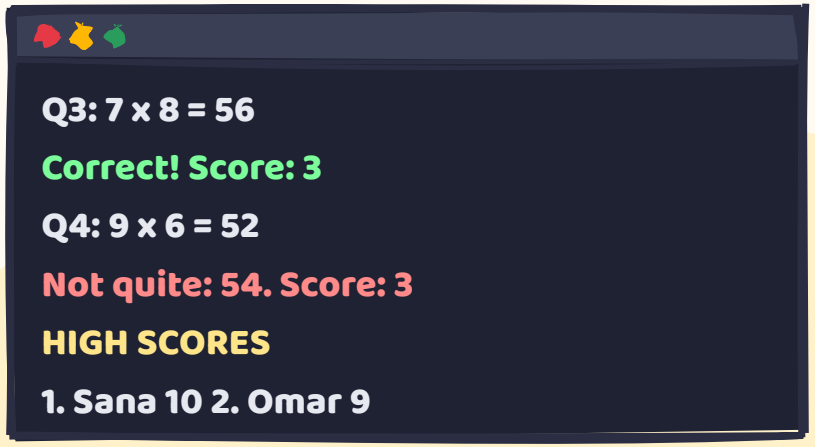
Young Innovators · Class 13

Python Maths Quiz with Scoreboard

Build a game your friends will want to beat!

Variables & loops | Functions | Saving to a file



Did you know?
Python is used by NASA, Instagram and YouTube, and it is the most popular language for AI.

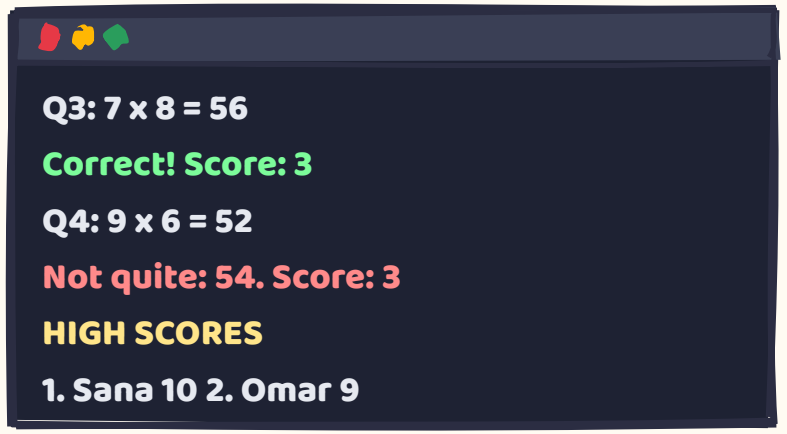
LEARN

Let's think...

Why are times tables boring?

The problem

Practising maths from a worksheet is dull. But give people a **score**, a **timer** and a **leaderboard**, and suddenly everyone wants to play again!



More worksheets?

Flash cards?

Make it a GAME!

Today: you code a maths quiz in **Python** that asks random questions, keeps score and saves a **high-score table**.

LEARN

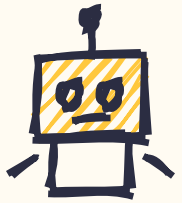
Learning aims

Today I will learn to...

1. Use **variables**, **input** and **if** statements
2. Repeat questions with a **for loop** and keep a score
3. Generate **random** questions and handle wrong typing
4. Organise code into **functions**
5. **Save** and **sort** high scores in a file

Keywords

variable · input · random · loop · function · try/except · file



LEARN

Watch and think

Video time!



Python in 100 Seconds

Fireship · about 3 minutes · youtu.be/x7X9w_GIm1s

Watch for...

1. Why is Python popular with beginners?
2. What does **indentation** do in Python?
3. Name one thing people build with Python.

Scan to watch

After the video: The video is fast! Pause and replay. Write down one Python word you will use in our quiz game.

Real world

Where is Python used?



AI

Most machine-learning tools, even ChatGPT-style models, are built with Python



Space

Scientists analyse telescope and rocket data



Games

Game tools, mods and simple games (like ours!)



Websites

Instagram and many others run Python on their servers

? Python reads almost like English. Why might that make it popular with beginners AND experts?

Fun fact

Python is named after a comedy show, not the snake!

LEARN

Python toolbox

Variables, input and if

```
name = input("What is your name? ")   # text from the keyboard
print("Good luck,", name)

score = 0                             # a variable: a labelled box
score = score + 1                     # now it holds 1

answer = int("42")                    # turn text into a number
if answer == 42:
    print("Correct!")
else:
    print("Wrong")
```

input() gives TEXT

Even if you type 42, Python sees the text **"42"**. Use **int()** to turn it into a number before comparing.

= vs ==

= puts a value IN a box. == ASKS if two things are equal.

LEARN

Bullet-proof input

try and except

```
def get_number(prompt):
    while True:
        reply = input(prompt)
        try:
            return int(reply)          # works for "42"
        except ValueError:
            print("Please type a whole number!")   # "abc" lands here
```

How it works

Python **tries** int(reply). If it fails with a **ValueError**, instead of crashing it runs the except part and asks again.

Use it

Replace **int(input(...))** in ask_question() with **get_number(...)**. Now nobody can crash your quiz!

ValueError
is the error name you saw in the bug report. Now you can catch it!

LEARN

Level 2

Difficulty levels

```
level = get_number("Level 1, 2 or 3? ")

if level == 1:
    top = 5            # easy: up to 5 x 5
elif level == 2:
    top = 12           # normal times tables
else:
    top = 20           # hard!

a = random.randint(1, top)
b = random.randint(1, top)
```

elif

Short for **else if**. Python checks each condition in order and runs the **first** one that is true.

More variety

Add a level for **+** and **-** questions too: pick the operation with **random.choice(["+", "-", "x"])**.

LEARN

Level 3

Add a timer and bonus points

```
import time

start = time.time()                  # seconds since 1970!
# ... ask all 10 questions ...
seconds = round(time.time() - start, 1)
print("You took", seconds, "seconds")

bonus = max(0, 60 - int(seconds))    # faster = more bonus points
total = score * 10 + bonus
```

How it works

Save the time at the **start**, then subtract it from the time at the **end**. The difference is how long the player took.

Fair scoring

Correct answers are worth **10** each. Speed only adds a **bonus**, so guessing fast doesn't win.

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Draw the game as a flowchart
2. One random question
3. 10 questions and a score
4. Functions: name a job once, use it many times
5. Break my program!
6. Save to a file
7. Read, sort and show the top 5

MAKE

Plan first

Draw the game as a flowchart



Why plan?

A flowchart shows every **step** and **decision** before you type. It is much easier to fix a drawing than to untangle code.

Your turn

Sketch YOUR game's flowchart. What extra boxes will a timer need?

Shapes

Rectangle = do something

Diamond = a yes/no question

MAKE

Version 1

One random question

```
import random

a = random.randint(1, 12)    # a whole number from 1 to 12
b = random.randint(1, 12)
correct = a * b

reply = input(f"{a} x {b} = ")
if int(reply) == correct:
    print("Correct!")
else:
    print("Not quite, it was", correct)
```

import random

Python comes with ready-made **modules**. **random** can roll dice, shuffle lists and pick numbers.

f-strings

f"{a} x {b} = " puts the values of a and b into the text. Neat!

Run it 5 times

Do you get the same question twice? That is randomness!

MAKE

Version 2

10 questions and a score

```
import random
score = 0

for q in range(1, 11):                # q = 1, 2, ... 10
    a = random.randint(1, 12)
    b = random.randint(1, 12)
    reply = input(f"Q{q}: {a} x {b} = ")
    if int(reply) == a * b:
        score = score + 1
        print("Correct! Score:", score)
    else:
        print("Not quite:", a * b)

print("Final score:", score, "out of 10")
```

for loop

range(1, 11) counts 1 to 10 (it stops **before** 11). Everything indented underneath repeats 10 times.

Indentation matters!

Python uses **4 spaces** to know what is inside the loop. Wrong spaces = wrong program.

MAKE

Tidy up

Functions: name a job once, use it many times

```
def ask_question(number):
    a = random.randint(1, 12)
    b = random.randint(1, 12)
    reply = input(f"Q{number}: {a} x {b} = ")
    if int(reply) == a * b:
        print("Correct!")
        return True
    print("Not quite:", a * b)
    return False

score = 0
for q in range(1, 11):
    if ask_question(q):
        score = score + 1
```

def = define

ask_question is a new command we invented. It takes a question number and **returns** True or False.

Why bother?

Shorter main code, easier to read, and you can change how questions work in **one place**.

Name it well

ask_question() says what it does. f1() says nothing!

MAKE

Class game

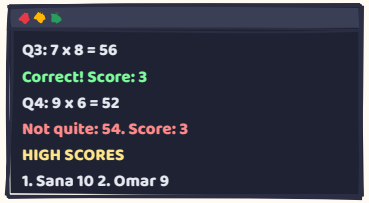
Break my program!

How to play

1. Swap laptops with another pair.
2. Your job: make their quiz **crash** (show a red error).
3. Try typing letters, nothing at all, spaces, or 3.5.
4. Write down exactly what you typed and the error message.
5. Swap back and read your bug report.

Then discuss

Which input crashed it? Why would a real game player do that?



Big idea

Users ALWAYS type unexpected things. Good programs expect the unexpected.

MAKE

Remember the scores

Save to a file

```
def save_score(name, total):
    with open("scores.txt", "a") as f:        # "a" = append
        f.write(f"{name},{total}\n")           # one line per game

# scores.txt now looks like:
# Sana,112
# Omar,95
# Ali,104
```

Why a file?

Variables are **forgotten** when the program ends. A file stays on the disk, so high scores survive until tomorrow.

"a" not "w"

"w" (write) wipes the file first! **"a"** (append) adds to the end.

Backslash n

means new line, so every game goes on its own line.

MAKE

The leaderboard

Read, sort and show the top 5

```
def show_high_scores():
    scores = []
    with open("scores.txt") as f:
        for line in f:
            name, total = line.strip().split(",")
            scores.append((int(total), name))
    scores.sort(reverse=True)              # biggest first
    print("=== HIGH SCORES ===")
    for place, (total, name) in enumerate(scores[:5], start=1):
        print(f"{place}. {name:10} {total}")
```

Tricks used

split(",") breaks a line in two.
(total, name) puts the score first, so **sort** orders by score.
[:5] takes the first 5.

No file yet?
The first time, scores.txt may not exist. Use try/except FileNotFoundError!

Test like an engineer

Playtest and feedback

Feedback (from another pair)	Notes
Game explains the rules clearly	_____
Could not crash it	_____
Difficulty felt fair	_____
Wanted to play again	_____
One idea to make it better	_____

Debug checklist

NameError? A variable is misspelled or used before it is set.
IndentationError? Check your 4 spaces.
Scores in a strange order? Did you convert to int?

Give feedback as: one thing that is GREAT, one thing to IMPROVE. Be kind and specific.

MAKE

Level up!

Quiz challenges

Choose your challenge

- **Mixed operations:** +, -, x and ÷ (only questions with whole-number answers!).
- **Learn from mistakes:** at the end, repeat every question the player got wrong.
- **Streaks:** 3 correct in a row gives double points.
- **Window game:** use the turtle or tkinter module to show questions in a window.

Think like an engineer

How could your quiz help a younger student practise exactly the tables they find hardest?

Boss level

Store the wrong answers per player in a file and ask those tables more often. That is adaptive learning, like real learning apps!

Show what you know

What did we learn?

- [x] input() gives text; int() turns it into a number
- [x] for loops repeat; if/elif/else decide
- [x] random makes every game different
- [x] Functions and try/except make code tidy and crash-proof
- [x] Files keep high scores after the program ends

At home

Challenge a family member to your quiz. Can they beat your high score?



Exit ticket: why must we use int() on the player's answer?

Next class:
Personal Portfolio Website!